Technical Answers – AIU Copilot Navigation Prototype

Zaynab Chaiba

1. Introduction

Dans le cadre de cet exercice technique, un prototype d'application web a été développé afin de visualiser la trajectoire d'un navire et certaines variables de navigation comme la vitesse ou le régime moteur (RPM).

Le fichier Excel fourni contient différentes séries de données liées à la navigation du navire sur une période d'environ dix mois, notamment les positions GPS, la vitesse, le cap, ainsi que certaines variables physiques et environnementales.

Les sections suivantes présentent :

- Une proposition d'architecture de base de données adaptée à ce type d'application,
- Une réflexion sur une fonctionnalité avancée permettant de modifier un segment de trajectoire directement sur la carte.

2. Réflexion sur l'architecture de la base de données

L'application AIU Copilot manipule des données spatio-temporelles provenant de plusieurs sources : positions GPS, variables de navigation, capteurs embarqués et conditions de mer. Le fichier Excel fourni illustre cette diversité avec plusieurs feuilles telles que *Ship Navigation*, *Ship Gyrocompass*, *Ship Hydro*, *Sea Conditions Prediction* ou encore *Sea Conditions Measured*.

Pour structurer efficacement ces données, une architecture reposant sur PostgreSQL/PostGIS et TileDB semble adaptée.

2.1. PostgreSQL / PostGIS

Dans le cadre du prototype, PostgreSQL/PostGIS constitue la base principale permettant de stocker les données de navigation et les géométries spatiales.

Les positions successives du navire peuvent être stockées sous forme de points géographiques, tandis que les trajectoires peuvent être représentées par des segments reliant deux positions successives.

Une organisation simple de la base pourrait être la suivante :

Table : vessels

Cette table contient les informations générales sur les navires.

Exemples de champs :

- vessel_id
- imo_code

- vessel_name

Dans le prototype utilisé pour l'exercice, les navires sont représentés par IMO1, IMO2 et IMO3.

Table : trajectory_points

Cette table stocke les positions successives du navire dans le temps.

Exemples de champs :

- point_id
- vessel_id
- timestamp
- geom (POINT)
- speed_kn
- course_deg
- heading_deg

Table : trajectory_segments

Cette table représente les segments reliant deux positions successives.

Exemples de champs :

- segment_id
- vessel_id
- time_start
- time_end
- geom (LINESTRING)
- speed_kn
- rpm

Les segments permettent notamment d'afficher la trajectoire sur la carte et de colorer chaque portion de trajectoire selon une variable choisie (par exemple la vitesse ou le RPM).

Table : environment_conditions

Certaines données environnementales peuvent être stockées dans une table spécifique.

Exemples de champs :

- condition_id
- timestamp

- vessel_id
- wave_height
- current_speed
- wind_direction

Ces informations peuvent provenir des feuilles Sea Conditions Prediction ou Sea Conditions Measured du fichier Excel.

2.2. Rôle de TileDB

Dans le prototype simplifié développé pour l'exercice, PostgreSQL/PostGIS est suffisant pour gérer les données. Cependant, TileDB devient particulièrement intéressant pour gérer des données volumineuses et multidimensionnelles, comme :

- Les données météorologiques globales,
- Les champs de vent ou de courant,
- Les séries temporelles issues de capteurs haute fréquence.

Dans une architecture complète de l'application, PostgreSQL/PostGIS pourrait donc gérer les données métier et géographiques, tandis que TileDB serait utilisé pour stocker des données environnementales massives indexées par temps et espace.

3. Fonctionnalité avancée : modification d'un segment de trajectoire

La fonctionnalité avancée consiste à permettre à l'utilisateur de modifier une trajectoire en déplaçant un point directement sur la carte.

3.1. Principe général

La trajectoire d'un navire est construite à partir d'une succession de points ordonnés dans le temps.
Les segments affichés sur la carte correspondent aux lignes reliant deux points consécutifs.

Par exemple :

segment 1 : P1 → P2
segment 2 : P2 → P3

Si l'utilisateur déplace le point **P2**, les segments adjacents doivent être recalculés :

P1 → P2'
P2' → P3

Seuls les segments voisins sont donc modifiés.

3.2. Interaction utilisateur

Le scénario pourrait être le suivant :

- L’utilisateur active le mode édition ;
- Il sélectionne un point de la trajectoire ;
- Le point devient déplaçable (draggable) ;
- L’utilisateur déplace ce point vers une nouvelle position ;
- Les segments voisins sont automatiquement recalculés et mis à jour sur la carte.

3.3. Logique de mise à jour des données

La modification repose sur la table des points de trajectoire, qui constitue la donnée de base.

Lorsqu’un point est déplacé :

- Ses nouvelles coordonnées remplacent les anciennes dans la table *trajectory_points* ;
- Les segments voisins sont recalculés dans *trajectory_segments.*

Certains indicateurs comme la distance parcourue ou la vitesse moyenne peuvent également être recalculés.

3.4. Intérêt de cette fonctionnalité

Cette fonctionnalité permet de simuler une modification de route ou d’explorer différents scénarios de navigation.
Dans une version plus avancée de l’application, elle pourrait être utilisée pour recalculer l’heure d’arrivée ou optimiser la trajectoire en fonction des conditions météorologiques.

4. Conclusion

L’architecture proposée repose sur une séparation entre les points de trajectoire, les segments et les données environnementales. PostgreSQL/PostGIS permet de gérer les données géographiques, tandis que TileDB peut être utilisé pour stocker des données spatio-temporelles volumineuses.

La fonctionnalité de modification d’un point de trajectoire permet d’imaginer des outils plus avancés pour l’analyse et l’optimisation de la navigation.